

Your first TensorStudio journey

You do not need to understand every Bittensor term before beginning. Start with one small question: **can we route a support message to the right team?**

Watch the five-minute walkthrough · Open TensorStudio

Video time	Step	What it means
00:00–00:50	Understand the problem	A service needs a task, software, evaluation and visibility
00:50–01:30	Play Bittensor Quest	Learn the words, check an answer and try a local coding exercise
01:30–02:05	Build a small interface	Ask for a plan and inspect a working support-inbox prototype
02:05–02:45	Define and test the task	Supply labelled examples, assemble files and inspect scoring
02:45–03:15	Run and inspect	Read miner logs, stop/start a process and run a console command
03:15–03:40	Read the network	Explore testnet events, a subnet dashboard and wallet tools
03:40–04:25	Meet TensorNode	Watch its existing zero-spend draft, events, billing and cancellation workflow
04:25–04:45	Find the source	Understand which project owns each part
04:45–05:00	Take the next step	Choose learning, development or operations

1. Learn the vocabulary through play

Open Bittensor Quest, select **Start Exploring**, and use arrows/WASD to move. The Learning panel offers six short lessons. For the first one:

1. Choose **TAO Fundamentals** and read the explanation.
2. Select your answer and press **Check Answer**.
3. Select **Start Code Challenge** and read the classroom task.
4. Complete `solution(input)` and choose **Run Local Tests**.
5. Open **Progress** to see the lesson's learning XP.

Quiz XP is a learning score, not real money or TAO. Code exercises are simplified simulations, not actual staking, transfers or consensus. Lessons and local tests work independently of the optional AI tutor.

2. Put a problem into words

Open Studio's **Playground** and explain the user, task and desired result. For example:

> Help a customer-support team route messages into billing or technical. What labelled data and scoring rule would make this checkable?

Review the response. Ask follow-up questions before accepting its assumptions. An app suggestion opens the interface builder; a subnet suggestion goes to subnet setup.

3. Build the interface around the service

Open **Subnet builder** and request a small support-inbox interface. Inspect the code and use its actual preview. In this film, the review buttons are human controls; they are not presented as live model predictions. A useful interface and a working miner model are separate pieces of the application.

4. Explain what a correct answer looks like

In **Subnet setup**, describe the task and paste reviewed JSONL examples. Each line has `input` and `answer`. For example format only:

```
{"input":"Please refund my invoice","answer":"billing"}
{"input":"The page crashes after login","answer":"technical"}
```

Generate a recipe and inspect the task, scoring and weight blocks. Choose **Assemble project and check code**. The files provide a concrete starting project; this action does not register a subnet.

5. Inspect the scoring

Choose **Run offline harness**, then **Open saved report**. Compare the controlled correct, incorrect, late and empty answers. Classification labels may match without implying plagiarism. Fixture scores are real calculations, while timing is simulated. They do not prove how an unseen model will perform.

6. Run with the right access

The hosted execution tools require the workspace owner's sign-in. A miner requires configured inference and valid public identity values. Open **Nodes**, inspect the actual process output, and use stop/start controls. Console commands stream real output from the connected runner.

The film also runs a development-only signed miner probe. It sends a new message to the generated miner and prints its actual model response. This optional helper is supplied separately from the generated templates and uses the public Alice development identity; it must not be used for valuable funds or as production authentication.

The hosted free runner can sleep and reset files; reassembly/relaunch may be needed afterward. Process startup is distinct from chain registration, serving and rewards.

7. Observe independently

Open **TensorScan**, choose the intended network and inspect blocks/events. Open a subnet dashboard to see observed participants. The example testnet subnet is public network state, not a claim that the demo owns it. Wallet tools are available separately; never record or share recovery material.

8. Review compute operations in TensorNode

Open the separate TensorNode repository and its full working demo. The combined film uses that real recording:

1. Save a zero-spend draft describing intended work.
2. Retrieve the persisted record.
3. Read stored events and follow event updates.
4. Inspect billing separately from deployment intent.
5. Cancel the unallocated draft and verify its saved state.

The first provider target is Lium SN51. This recorded workflow does not show a paid GPU allocation or successful GPU execution. TensorNode has its own Rust/Next.js/PostgreSQL setup; it is not an already integrated Studio service.

9. Find the implementation

The ecosystem guide links Studio, TensorNode and Subnet Factory source. Factory is a separate miner-contribution generation prototype; the current evidence does not establish a registered public Factory subnet serving Studio's assistant. Private repositories require access from their owners.